

ECON1059 Seminar 7

Notes for the Seminar Series on Python Programming for Economics (ECON1059, Fall 2025)

Juncheng ZHANG, PhD Candidate in Economics, UNNC.

Research interest: Behavioral and Experimental Economics.

Email: Juncheng.ZHANG@nottingham.edu.cn

Classes

A class puts together:

- data (stored in each instance)
- methods (functions acting on that data)

This lets you build many objects with similar structure but different values (e.g., different firms or consumers), helping code stay clean and organized.

Basic structure:

```
class ClassName:
    """
    Optional: Class docstring
    Describe what this class represents and what it is used for.
    """

    # Constructor
    # Initializes instance attributes
    def __init__(self, arg1, arg2, ...):
        self.attr1 = arg1      # instance attribute
        self.attr2 = arg2
        # ...

    # Instance method
```

```

# Behavior acting on instance data
def method1(self, ...):
    # access instance data via self
    # do something
    return ...

# Optional: String representation
# What print(obj) shows
def __str__(self):
    return f"ClassName(attr1={self.attr1}, attr2={self.attr2}, ...)"

```

Simple Classes

- Experiment with classes by creating any object you find interesting. Afterwards, create a parent class.
 - potential examples: a person, a student; a motor-vehicle, a car, a bus; a video game weapon, a video game sword (or something less violent); an economic agent, a consumer; animals; flowers, etc.
 - for potential methods, think of what these objects do (methods = verbs), e.g., a dog barks, a car accelerates or turns, a weapon shoots.
 - If you're not very sure about your solutions, show it to a classmate or our TAs. You can also search online. Unless you were crazily creative, you'll be able to find an example

In [1]: *## Example 1: Class and instances (Part of ans to Q1)*

```

class Pet:
    # Instance attributes(Constructor): data stored in each object
    # Every method in a class is written from the object's point of view.
    # self means "me", the object being created
    # So self.name = my own name, self.age = my own age.
    def __init__(self, name, age, gender, species):
        self.name = name
        self.age = age
        self.gender = gender
        self.species = species

    # Instance method: returns a text describing this dog.
    # self refers to the object that is calling this method OR self = the object calling this method
    def introduce(self):
        """Return basic intro text."""
        return f"My name is {self.name}. I am a {self.age}-year-old {self.gender} {self.species}."

    # __str__: representation when printed

```

```

def __str__(self):
    return (
        f"Pet(name={self.name}, "
        f"species={self.gender}, "
        f"age={self.age}, "
        f"species={self.species})"
    )

# When defining an instance
# → Python runs: Pet.__init__(my_pet, "Siri", 2, "male", "dog")
# self.name = "Siri"
# self.age = 2
# self.gender = "male"
# self.species = "dog"
# my_pet is a variable that stores this new object.
# Inside __init__, self refers to that same object (not the variable name my_pet).
# my_pet → Object ← self
my_pet = Pet("Siri", 2, "male", "dog")

# When calling introduce():
# → Python runs: Pet.introduce(my_pet) → self receives the object stored inside my_pet
print(my_pet.introduce())

print(my_pet)
print("")
marcus_dog = Pet("Rhuby", 1, "female", "dog")
print(marcus_dog.introduce())
print(marcus_dog)

```

My name is Siri. I am a 2-year-old male dog.
 Pet(name=Siri, species=male, age=2, species=dog)

My name is Rhuby. I am a 1-year-old female dog.
 Pet(name=Rhuby, species=female, age=1, species=dog)

Inheritance

Inheritance allows a class (child/subclass) to reuse and extend the properties and methods of another class (parent/superclass).

- Parent → Child structure (The child automatically gets attributes(methods) defined by the parent)

```
class Parent:
```

```
    ...
```

```
class Child(Parent):
```

```
...
```

- Reuse parent initialization: `super()`

```
class Rectangle:
```

```
    def __init__(self, width, height):  
        self.width = width  
        self.height = height
```

```
class Square(Rectangle):
```

```
    def __init__(self, length):  
        super().__init__(length, length)  # call parent __init__
```

- Child can add new features (even override)

```
In [2]: # Example 2 Dog class Inheriting from Pet (Part of ans to Q1)  
class Dog(Pet):  
    """  
    Dog class inherits from Pet.  
    Adds breed information.  
    """  
    def __init__(self, name, breed, age, gender):  
        # Call parent constructor, species always 'dog'  
        super().__init__(name, age, gender, species="dog")  
        self.breed = breed  
  
    # Override introduce to include breed  
    def introduce(self):  
        return (  
            f"My name is {self.name}. "  
            f"I am a {self.age}-year-old {self.gender} {self.breed}."  
        )  
  
    def bark(self):  
        return f"{self.name} says: Woof! Woof!"  
  
marcus_dog = Dog("Rhuby", "von Rhubarb", 1, "female")  
my_dog      = Dog("Siri", "Shiba Inu", 4, "male")  
  
print(marcus_dog.introduce())  
print(marcus_dog.bark())  
print("")
```

```
print(my_dog.introduce())
print(my_dog.bark())
```

My name is Rhuby. I am a 1-year-old female von Rhubarb.
Rhuby says: Woof! Woof!

My name is Siri. I am a 4-year-old male Shiba Inu.
Siri says: Woof! Woof!

`dir()`, classes, and attributes

- Use your answer above or from our lectures to explore how classes, attributes, and the `dir()` function works. Do your class's attributes (defined in `__init__()`) show up when you print `dir(class_instance_name)`? Do they show up when you call `dir(class_name)`? Can you think of why this is the case?
- Since attributes are created in the `__init__()` method, i.e., upon the creation of an instance, they don't show up when you call `dir(class_name)` but do when you execute `dir(class_instance_name)`
- See examples below

```
In [3]: # No attributes shown:
print(dir(Pet))
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__firstlineno__', '__format__', '__ge__', '__getattr__',
 '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__',
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__static_attributes__', '__str__', '__subclasshook__', '__weakref__',
 'introduce']
```

```
In [4]: # Attributes shown:
print(dir(my_dog))
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__firstlineno__', '__format__', '__ge__', '__getattr__',
 '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__',
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__static_attributes__', '__str__', '__subclasshook__', '__weakref__',
 'age', 'bark', 'breed', 'gender', 'introduce', 'name', 'species']
```

Playing with Rectangles

- Create a rectangle class that includes a method that cuts the rectangle in half (along the largest side)
 - Can you write the cut-method so that it returns the cut-away part of the rectangle as an object itself
 - Finally, write a function (using the above class) to calculate the sum of the areas of the following objects: (the area of) an initial square, (the area of) the square cut in half, (the area) of the cut-in-half square cut in half, ..., up to n-cuts! Needless to say, this function should use your class-objects (not your

advanced understanding of math).

- **Note on how to tackle this question:** while the above question text guides you towards (what I think) is a very cool implementations, there are other solutions that can solve this problem just as well. If you find the above hard to implement, try a different approach.

```
In [5]: class RectangleCutable:
        """ A Rectangle that can be cut in half along the longest edge.

        Attributes:
            width: the width of the rectangle
            height: the height of the rectangle
        """

        def __init__(self, width, height):
            self.width = width
            self.height = height

        def get_area(self):
            return self.width * self.height

        def cut_in_half(self):
            """ Cut the current rectangle in half
            return: cut-away part of rectangle as a new Object.
            """

            if self.width >= self.height:
                self.width = self.width / 2
            else:
                self.height = self.height / 2

            return RectangleCutable(self.width, self.height)

        def __str__(self):
            return f"Rectangle of width = {self.width} and height = {self.height}."
```

```
In [6]: # Testing RectangleCutable
        rectangle_0 = RectangleCutable(6, 8)
        print(f"Original rectangle: {rectangle_0}")
```

Original rectangle: Rectangle of width = 6 and height = 8.

```
In [7]: rectangle_1 = rectangle_0.cut_in_half()
        print(f"New Rectangle, after one round of cutting:\n {rectangle_1}")
```

New Rectangle, after one round of cutting:
Rectangle of width = 6 and height = 4.0.

```
In [8]: print(f"Old Rectangle, after one round of cutting:\n {rectangle_0}")
```

Old Rectangle, after one round of cutting:
Rectangle of width = 6 and height = 4.0.

Solution to:

- Calculate the area of the series of n rectangles where the next rectangle is half of the previous.

```
In [9]: def find_area_series_cut_rectangles(initial_height, initial_width, num_cuts):  
        rectangle = RectangleCuttable(initial_height, initial_width)  
        area = rectangle.get_area()  
        for i in range(num_cuts):  
            rectangle = rectangle.cut_in_half()  
            area += rectangle.get_area()  
        return area
```

```
In [10]: # Test find_area_series_cut_rectangles  
height = 1  
width = 1  
print(f"Area of a series of rectangles, with the first being a 1x1 square, the scond a 0.5x1 rectangle is:")  
print(find_area_series_cut_rectangles(height, width, 1))  
  
print(f"For: 1x1, 0.5x1, and 0.5x0.5, we have:")  
print(find_area_series_cut_rectangles(height, width, 3))
```

Area of a series of rectangles, with the first being a 1x1 square, the scond a 0.5x1 rectangle is:
1.5
For: 1x1, 0.5x1, and 0.5x0.5, we have:
1.875

- You could manually check whether these are right or use the classic formula that calculates $1 + 1/2 + 1/4 + \dots + (1/2)^n$

@staticmethod

A static method is a function defined inside a class that:

- does not take `self`
- does not use instance attributes

It behaves like a **normal function**, but is grouped inside the class for organization.

```
In [11]: ## Example 3: Dog class with static method
class Dog(Pet):
    """
    Dog class inherits from Pet.
    Adds breed information and dog-specific behavior.
    """

    # Constructor
    # name, breed, age, gender → passed in
    def __init__(self, name, breed, age, gender):
        # Call parent(Pet)'s constructor
        super().__init__(name, age, gender, species="dog")
        self.breed = breed # new attribute only in Dog

    # Instance method: needs self
    def bark(self):
        return "Woof! Woof!"

    # Static method: does NOT take self, universal dog sound
    @staticmethod
    def bark_static():
        return "Woof! Woof!"
```

```
In [12]: # Create a Dog instance
my_dog = Dog("Siri", "Shiba Inu", 4, "male")

# When calling introduce():
# → Python runs: Pet.introduce(my_dog) → self = my_dog
print(my_dog.introduce()) # ← inherited from Pet
print(my_dog.bark())      # ← Dog instance method, we know who is barking – Siri
print(my_dog)             # __str__ from parent Pet

print("")
# Static method behaves like a normal function placed inside the class
print(Dog.bark_static())  # ← static method, we don't know who is barking, just returns a sound
```

My name is Siri. I am a 4-year-old male dog.
Woof! Woof!
Pet(name=Siri, species=male, age=4, species=dog)

Woof! Woof!

- Write a class with a staticmethod. Next write the same staticmethod also a normal method, passing in the `self` tag but never using it inside.
 - Try calling both methods after instantializing a version of the class. Try also calling both methods without creating an instance.


```
In [13]: # Staticmethod homework

class StaticMethodExample:
    @staticmethod
    def add_two_numbers_static(a, b):
        return a + b

    def add_two_numbers(self, a, b):
        return a + b

    # What happens if we also add a method without self but don't call it static?
    def add_two_numbers_no_self(a, b):
        return a + b
```

```
In [14]: class_static = StaticMethodExample()
a = 5
b = 6
```

```
In [15]: # Calling a method via the instance of the class works for both
print(f"Add via staticmethod with instance. a = {a}, b = {b}")
print(f"a + b = {class_static.add_two_numbers_static(a, b)}")

print(f"Add via normal method with instance. a = {a}, b = {b}")
print(f"a + b = {class_static.add_two_numbers(a, b)}")

# Calling a method via the class only works for the static method
print(f"Add via staticmethod with class. a = {a}, b = {b}")
print(f"a + b = {StaticMethodExample.add_two_numbers_static(a, b)}")
```

```
Add via staticmethod with instance. a = 5, b = 6
a + b = 11
Add via normal method with instance. a = 5, b = 6
a + b = 11
Add via staticmethod with class. a = 5, b = 6
a + b = 11
```

```
In [16]: # raises Error: calling non-static method using class object.
# print(f"Add via staticmethod with class. a = {a}, b = {b}, a + b = {StaticMethodExample.add_two_numbers(a, b)}")

# raises Error as well: non-static but without self
# print(f"Add via no-self method with instance. a = {a}, b = {b}, a + b = {class_static.add_two_numbers_no_self(a, b)}")

# Whereas non-static method without a self works when we call it via the class object itself
print(f"Add via no-self method with class. a = {a}, b = {b}")
print(f"a + b = {StaticMethodExample.add_two_numbers_no_self(a, b)}")
```

Add via no-self method with class. `a = 5, b = 6`
`a + b = 11`

Methods with `self` → can only be called using an instance

Methods with `@staticmethod` → can be called using either the class or an instance

Methods without `self` and without `@staticmethod` → can only be called using the class

- TAKEAWAY: use `@staticmethod` to get it right in all cases :).

Inheritance

- In the class `square` of our lecture notes, we called `super().__init__(length, length)` in the `__init__` method. How could you write the class without relying on the parent's initialization?
(If you get it right on the first try, also try to write a version that doesn't work to make sure whether you really understand it).
- Shortest solution (all pointless stuff cut)

```
In [17]: class Rectangle:
        def __init__(self, width, height):
            self.width = width
            self.height = height

        def get_area(self):
            return self.width * self.height

        class Square(Rectangle):
            def __init__(self, length): #override the initializer
                # super().__init__(length, length)
                self.width = length
                self.height = length

            def get_diagonal(self):
                return (2*self.width**2)**0.5
```

```
In [18]: my_square = Square(4)
        print(f"Area = {my_square.get_area()}")
```

Area = 16

- The following implementation does not work

```
In [19]: class SquareBroken(Rectangle):
        def __init__(self, length):
            self.length = length

        def get_diagonal(self):
            # Notice the change to length. This will work!
            return (2*self.length**2)**0.5
```

```
In [20]: my_broken_square = SquareBroken(4)
        print(f"Diagonal = {my_square.get_diagonal()}")
```

Diagonal = 5.656854249492381

```
In [21]: # In class SquareBroken, we did not set width and height attributes.
        # So, when we call get_area(), it will raise an AttributeError.
        print(f"Area = {my_broken_square.get_area()}")
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[21], line 3
      1 # In class SquareBroken, we did not set width and height attributes.
      2 # So, when we call get_area(), it will raise an AttributeError.
----> 3 print(f"Area = {my_broken_square.get_area()}")

Cell In[17], line 7, in Rectangle.get_area(self)
      6 def get_area(self):
----> 7     return self.width * self.height

AttributeError: 'SquareBroken' object has no attribute 'width'
```